

Asian School of Management & Technology

(Affiliated to Tribhuvan University)

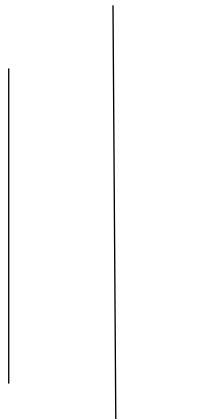
Gongabu, Kathmandu

Web: www.asm.edu.np



Lab Report

Advanced Java Programming



Submitted By:

Sushank Lamsal

B.Sc.CSIT – 7th Semester

Symbol No : 79010980

Submitted To:

Mr. Prabin Silwal

Asian School of Management
& Technology

TABLE OF CONTENTS

Practical	S.No	Topic
Practical 1	Core Java Concepts	
	1	Class, Instance Variable, Constructor and Method
	2	One Dimensional and Multidimensional Array
	3	Method Overloading
	4	Single and Multi-level Inheritance
	5	Single Inheritance (Two Child Classes)
	6	Multi-level Inheritance
	7	Super Class Constructor
	8	Method Overriding
	9	Dynamic Method Dispatch
	10	Multiple Inheritance using Interface
	11	Interface Inheritance
	12	Multithreading using Thread Class and Runnable Interface
	13	Inter Thread Communication
	14	File Handling - Reading and Writing
15	Serialization and Deserialization	

Practical 2	Swing GUI Components	
	16	Complete Swing Form
	17	Layout Managers
	18	Four Buttons and Text Field
	19	JTable and JComboBox
	20	Swing Menu, Dialog, File Chooser, Color Chooser and Tree
Practical 3	Event Handling	
	21	Action Event using Radio Button and Normal Button
	22	Item Event using Check Box
	23	Handle Mouse Event
	24	Handle Key Event
	25	List Selection Event
	26	Focus Event
	27	Window Event using Adapter and Delegation Method
	28	Sum and Difference using Mouse Event

Practical 4	JDBC and Database	
	29	CRUD Operation using Statement
	30	CRUD Operation using PreparedStatement
	31	RowSet and Scrollable Updatable ResultSet
	32	Complete Registration and Login Form with Database
Practical 5	Networking	
	33	TCP Client-Server Application
	34	UDP Client-Server - Even or Odd
Practical 6	JavaFX	
	35	JavaFX Registration Form with Database
	36	JavaFX Layout Demo
Practical 7	JSP	
	37	JSP Registration, Login, Dashboard with Session

1. Write a program to create a class, instance variable, constructor and method. After this create a class FirstDemo with main method and access the member of above class.

Program:

```
class Student {  
    // Instance variables  
    String name;  
    String course;  
  
    // Constructor  
    Student(String n, String c) {  
        name = n;  
        course = c;  
    }  
  
    // Method  
    void display() {  
        System.out.println("Name: " + name);  
        System.out.println("Course: " + course);  
    }  
}  
  
public class FirstDemo {  
    public static void main(String[] args) {  
        Student s1 = new Student("Sushank Lamsal", "Bsc.CSIT");  
        s1.display();  
    }  
}
```

Output:

```
vscode → ../Sushank/java/Lab1-15/src $ java FirstDemo.java  
Name: Sushank Lamsal  
Course: Bsc.CSIT
```

2. Write a program to create one dimensional and multidimensional array and display the array using for and for each loop. All the data of array should be taken from user.

Program:

```
import java.util.Scanner;

public class Ques2 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // 1d array
        System.out.print("Enter size of 1D array: ");
        int n = sc.nextInt();

        int[] arr = new int[n];

        System.out.println("Enter elements of 1D array:");
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // for Loop
        System.out.println("\n1D Array using for loop:");
        for (int i = 0; i < n; i++) {
            System.out.print(arr[i] + "\t");
        }

        // for-each Loop
        System.out.println("\n1D Array using for-each loop:");
        for (int num : arr) {
            System.out.print(num + "\t");
        }

        // 2d array
        System.out.print("\n\nEnter rows of 2D array: ");
        int r = sc.nextInt();

        System.out.print("Enter columns of 2D array: ");
        int c = sc.nextInt();

        int[][] arr_2d = new int[r][c];
        System.out.println("Enter elements of 2D array:");
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                arr_2d[i][j] = sc.nextInt();
            }
        }
    }
}
```

```

    }
}

// for Loop
System.out.println("\n2D Array using for loop:");
for (int row = 0; row < arr_2d.length; row++) {
    for (int col = 0; col < arr_2d[row].length; col++) {
        System.out.print(arr_2d[row][col] + "\t");
    }
    System.out.println();
}

// for-each Loop
System.out.println("\n2D Array using for-each loop:");
for (int[] row : arr_2d) {
    for (int value : row) {
        System.out.print(value + "\t");
    }
    System.out.println();
}
sc.close();
}
}

```

Output:

```

vscode → ../Sushank/Java/Lab1-15/src $ java Ques2.java
Enter size of 1D array: 3
Enter elements of 1D array:
45
55
65

1D Array using for loop:
45    55    65
1D Array using for-each loop:
45    55    65

Enter rows of 2D array: 2
Enter columns of 2D array: 2
Enter elements of 2D array:
100
200
300
400

2D Array using for loop:
100    200
300    400

2D Array using for-each loop:
100    200
300    400

```

3. Write a program to demonstrate method overloading condition.

Program:

```
public class Ques3 {  
    static void add(int x, int y) {  
        System.out.println("Sum of 2 integers: " + (x + y));  
    }  
  
    static void add(String x, String y) {  
        System.out.println("Sum of 2 strings: " + (x + y));  
    }  
  
    public static void main(String[] args) {  
        add(10, 20);  
        add("Hello, ", "World!");  
    }  
}
```

Output:

```
● vscode → ../Sushank/Java/Lab1-15/src $ java Ques3.java  
Sum of 2 integers: 30  
Sum of 2 strings: Hello, World!
```

4. Write a program to demonstrate single level and multi-level inheritance.

Program:

```
// Single and Multilevel Inheritance  
  
//single  
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}  
  
// Multilevel  
class GrandFather {  
    void house() {  
        System.out.println("GrandFather has a house");  
    }  
}  
  
class Father extends GrandFather {  
    void bike() {  
        System.out.println("Father has a bike");  
    }  
}  
  
class Child extends Father {  
    void mobile() {  
        System.out.println("Child has a mobile");  
    }  
}  
  
public class Ques4 {
```

```

public static void main(String[] args) {

    // single level inheritace
    Dog dog = new Dog();
    dog.sound();
    dog.bark();

    // Multilevel inheritance
    Child c = new Child();
    c.house();
    c.bike();
    c.mobile();
}
}

```

Output:

```

● vscode → .../Sushank/Java/Lab1-15/src $ java Ques4.java
Animal makes sound
Dog barks
GrandFather has a house
Father has a bike
Child has a mobile

```

5. Provide an example of Single inheritance of your choice

Program:

// Single Inheritance Example

```

class BankAccount {
    String accountHolder = "Sushank";
    double balance = 5000;

    void showBalance() {
        System.out.println(accountHolder + " has balance Rs. " +
balance);
    }
}

class SavingsAccount extends BankAccount {
    double interest = 250;

    void addInterest() {
        balance = balance + interest;
        System.out.println("Interest added: Rs. " + interest);
    }
}

public class Ques5 {
    public static void main(String[] args) {
        SavingsAccount account = new SavingsAccount();

        account.showBalance();
        account.addInterest();
        account.showBalance();
    }
}

```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques5.java
Sushank has balance Rs. 5000.0
Interest added: Rs. 250.0
Sushank has balance Rs. 5250.0
```

6. Provide an example of multi-level inheritance of your choice

Program:

```
// Multilevel Inheritance Example

class Person {
    void showName() {
        System.out.println("Name: Ram");
    }
}

class Teacher extends Person {
    void showJob() {
        System.out.println("Job: Teacher");
    }
}

class MathTeacher extends Teacher {
    void showSubject() {
        System.out.println("Subject: Mathematics");
    }
}

public class Ques6 {
    public static void main(String[] args) {
        MathTeacher teacher = new MathTeacher();

        teacher.showName();
        teacher.showJob();
        teacher.showSubject();
    }
}
```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques6.java
Name: Ram
Job: Teacher
Subject: Mathematics
```

7. Provide example on how constructor of super class is called.

Program:

// Constructor of Super Class

```
class Parent {
    Parent() {
        System.out.println("Parent class constructor is called");
    }
}

class Child extends Parent {
    Child() {
        super();
        System.out.println("Child class constructor is called");
    }
}

public class Ques7 {
    public static void main(String[] args) {
        Child c = new Child();
    }
}
```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques7.java
Parent class constructor is called
Child class constructor is called
```

8. Provide example on method overriding condition

Program:

// Method Overriding Example

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Bike extends Vehicle {
    @Override
    void start() {
        System.out.println("Bike starts");
    }
}

public class Ques8 {
    public static void main(String[] args) {
        Bike bike = new Bike();
        bike.start();
    }
}
```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques8.java
Bike starts
```

9. Provide example on dynamic method dispatch.

Program:

```
// Dynamic Method Dispatch Example

class Food {
    void taste() {
        System.out.println("Food has different tastes");
    }
}

class Pizza extends Food {
    void taste() {
        System.out.println("Pizza tastes cheesy");
    }
}

class Mango extends Food {
    void taste() {
        System.out.println("Mango tastes sweet");
    }
}

public class Ques9 {
    public static void main(String[] args) {
        Food food;

        food = new Pizza();
        food.taste();

        food = new Mango();
        food.taste();
    }
}
```

Output:

```
● vscode → ../Sushank/Java/Lab1-15/Src $ java Ques9.java
Pizza tastes cheesy
Mango tastes sweet
```

10. Demonstrate multiple inheritance using interface.

Program:

```
// Multiple Inheritance Using Interface

interface Printable {
    void print();
}

interface Showable {
    void show();
}

class StudentRecord implements Printable, Showable {
    public void print() {
        System.out.println("Printing student record");
    }
}
```

```

    }

    public void show() {
        System.out.println("Showing student name: Harry");
    }
}

public class Ques10 {
    public static void main(String[] args) {
        StudentRecord student = new StudentRecord();

        student.print();
        student.show();
    }
}

```

Output:

```

● vscode → ../Sushank/Java/Lab1-15/Src $ java Ques10.java
Printing student record
Showing student name: Harry

```

11. Demonstrate inheritance in interface (extending the interface).

Program:

```

// Interface Extending

interface BasicPhone {
    void call();
}

interface SmartPhone extends BasicPhone {
    void internet();
}

class MyPhone implements SmartPhone {
    public void call() {
        System.out.println("Calling a friend");
    }

    public void internet() {
        System.out.println("Using internet on phone");
    }
}

public class Ques11 {
    public static void main(String[] args) {
        MyPhone phone = new MyPhone();

        phone.call();
        phone.internet();
    }
}

```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques11.java
Calling a friend
Using internet on phone
```

12. Write a program to demonstrate multithreading using both Thread class and Runnable interface.

Program:

```
// Multithreading Using Thread Class and Runnable Interface

class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Thread class: " + i);
        }
    }
}

class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Runnable interface: " + i);
        }
    }
}

public class Ques12 {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        Thread t2 = new Thread(new MyRunnable());

        t1.start();
        t2.start();
    }
}
```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques12.java
Thread class: 1
Thread class: 2
Thread class: 3
Runnable interface: 1
Runnable interface: 2
Runnable interface: 3
```

13. Write a program to demonstrate inter thread communication.

Program:

// Inter Thread Communication

```
class MessageBox {
    String message;
    boolean hasMessage = false;

    synchronized void writeMessage(String text) {
        message = text;
        hasMessage = true;
        System.out.println("Writer wrote a message");
        notify();
    }

    synchronized void readMessage() {
        while (!hasMessage) {
            try {
                wait();
            } catch (InterruptedException e) {
                System.out.println("Reader interrupted");
            }
        }

        System.out.println("Reader received: " + message);
    }
}

class Writer extends Thread {
    MessageBox box;

    Writer(MessageBox box) {
        this.box = box;
    }

    public void run() {
        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Writer interrupted");
        }

        box.writeMessage("Hello from writer thread");
    }
}

class Reader extends Thread {
    MessageBox box;

    Reader(MessageBox box) {
        this.box = box;
    }

    public void run() {
        box.readMessage();
    }
}

public class Ques13 {
```

```

    public static void main(String[] args) {
        MessageBox box = new MessageBox();

        Reader reader = new Reader(box);
        Writer writer = new Writer(box);

        reader.start();
        writer.start();
    }
}

```

Output:

```

● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques13.java
Writer wrote a message
Reader received: Hello from writer thread

```

14. Write a program to demonstrate reading and writing data in file.

Program:

```

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;

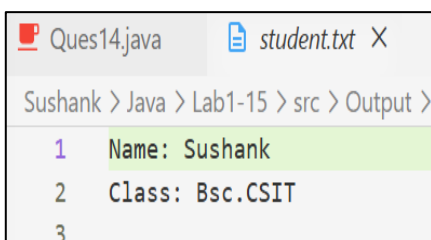
public class Ques14 {
    public static void main(String[] args) {
        try {
            FileWriter writer = new FileWriter("Output/student.txt");
            writer.write("Name: Sushank\n");
            writer.write("Class: Bsc.CSIT\n");
            writer.close();

            FileReader reader = new FileReader("Output/student.txt");
            int ch;

            System.out.println("Data from file:");
            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
            reader.close();
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}

```

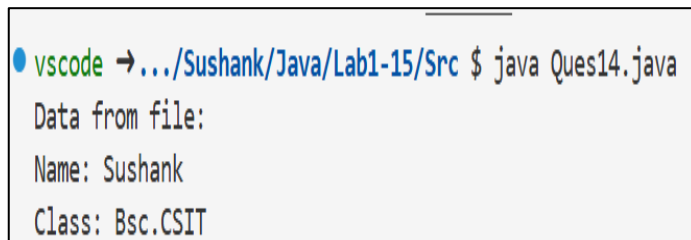
Output:



```

Ques14.java student.txt X
Sushank > Java > Lab1-15 > src > Output >
1 Name: Sushank
2 Class: Bsc.CSIT
3

```



```

● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques14.java
Data from file:
Name: Sushank
Class: Bsc.CSIT

```

15. Write a program to demonstrate serialization and deserialization.

Program:

```
import java.io.*;

// Serialization and Deserialization
class Student implements Serializable {
    int rollNo;
    String name;

    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }
}

public class Ques15 {
    public static void main(String[] args) {
        Student student = new Student(1, "Ram");

        try {
            // serialization
            ObjectOutputStream out = new ObjectOutputStream(new
FileOutputStream("Output/student.ser"));
            out.writeObject(student);
            out.close();

            // deserialization
            ObjectInputStream in = new ObjectInputStream(new
FileInputStream("Output/student.ser"));
            Student savedStudent = (Student) in.readObject();
            in.close();

            System.out.println("Student data after deserialization:");
            savedStudent.display();
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println("Class not found");
        }
    }
}
```

Output:

```
● vscode → .../Sushank/Java/Lab1-15/Src $ java Ques15.java
Student data after deserialization:
Roll No: 1
Name: Ram
```

Chapter 2

16. Write a program to demonstrate complete form using swing. Form should contain labels, text field, radio button, check box, combo box, text area and buttons. If button is pressed then display the content of form in label.

Program:

```
import javax.swing.*;

public class SwingForm_16 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Student Form");
        frame.setSize(520, 620);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel titleLabel = new JLabel("Student Form");
        titleLabel.setBounds(190, 20, 150, 25);
        frame.add(titleLabel);

        JLabel nameLabel = new JLabel("Name:");
        nameLabel.setBounds(50, 70, 100, 25);
        frame.add(nameLabel);

        JTextField nameTextField = new JTextField();
        nameTextField.setBounds(160, 70, 220, 25);
        frame.add(nameTextField);

        JLabel genderLabel = new JLabel("Gender:");
        genderLabel.setBounds(50, 115, 100, 25);
        frame.add(genderLabel);

        JRadioButton maleRadioButton = new JRadioButton("Male");
        maleRadioButton.setBounds(160, 115, 80, 25);
        frame.add(maleRadioButton);

        JRadioButton femaleRadioButton = new JRadioButton("Female");
        femaleRadioButton.setBounds(250, 115, 90, 25);
        frame.add(femaleRadioButton);

        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(maleRadioButton);
        genderGroup.add(femaleRadioButton);

        JLabel hobbyLabel = new JLabel("Hobbies:");
        hobbyLabel.setBounds(50, 160, 100, 25);
        frame.add(hobbyLabel);

        JCheckBox readingCheckBox = new JCheckBox("Reading");
        readingCheckBox.setBounds(160, 160, 90, 25);
        frame.add(readingCheckBox);

        JCheckBox musicCheckBox = new JCheckBox("Music");
        musicCheckBox.setBounds(260, 160, 80, 25);
        frame.add(musicCheckBox);

        JCheckBox sportsCheckBox = new JCheckBox("Sports");
        sportsCheckBox.setBounds(350, 160, 80, 25);
        frame.add(sportsCheckBox);
    }
}
```

```

JLabel courseLabel = new JLabel("Course:");
courseLabel.setBounds(50, 205, 100, 25);
frame.add(courseLabel);

String[] courses = {"BCA", "BBS", "BIM", "CSIT"};
JComboBox<String> courseComboBox = new JComboBox<>(courses);
courseComboBox.setBounds(160, 205, 220, 25);
frame.add(courseComboBox);

JLabel addressLabel = new JLabel("Address:");
addressLabel.setBounds(50, 250, 100, 25);
frame.add(addressLabel);

JTextArea addressTextArea = new JTextArea();
JScrollPane addressScrollPane = new JScrollPane(addressTextArea);
addressScrollPane.setBounds(160, 250, 220, 80);
frame.add(addressScrollPane);

JButton submitButton = new JButton("Submit");
submitButton.setBounds(160, 355, 100, 30);
frame.add(submitButton);

JButton clearButton = new JButton("Clear");
clearButton.setBounds(280, 355, 100, 30);
frame.add(clearButton);

JLabel resultLabel = new JLabel("Form details will be displayed
here.");
resultLabel.setBounds(50, 410, 420, 130);
resultLabel.setVerticalAlignment(SwingConstants.TOP);
frame.add(resultLabel);

submitButton.addActionListener(e -> {
    String gender = "Not selected";
    if (maleRadioButton.isSelected()) {
        gender = "Male";
    } else if (femaleRadioButton.isSelected()) {
        gender = "Female";
    }

    String hobbies = "";
    if (readingCheckBox.isSelected()) {
        hobbies = hobbies + "Reading ";
    }
    if (musicCheckBox.isSelected()) {
        hobbies = hobbies + "Music ";
    }
    if (sportsCheckBox.isSelected()) {
        hobbies = hobbies + "Sports ";
    }
    if (hobbies.equals("")) {
        hobbies = "None";
    }

    String address = addressTextArea.getText().replace("\n",
"<br>");

    resultLabel.setText("<html>"
        + "Name: " + nameTextField.getText() + "<br>"
        + "Gender: " + gender + "<br>"
        + "Hobbies: " + hobbies + "<br>"

```

```

        + "Course: " + courseComboBox.getSelectedItem() +
" <br>"
        + "Address: " + address
        + "</html>");
    });

    clearButton.addActionListener(e -> {
        nameTextField.setText("");
        genderGroup.clearSelection();
        readingCheckBox.setSelected(false);
        musicCheckBox.setSelected(false);
        sportsCheckBox.setSelected(false);
        courseComboBox.setSelectedIndex(0);
        addressTextArea.setText("");
        resultLabel.setText("Form details will be displayed here.");
    });

    frame.setVisible(true);
}
}

```

Output:

The screenshot shows a Java Swing window titled "Student Form". Inside the window, there is a form with the following fields and controls:

- Name:** A text field containing "Sushank".
- Gender:** Two radio buttons, "Male" (selected) and "Female".
- Hobbies:** Three checkboxes, "Reading" (unchecked), "Music" (checked), and "Sports" (checked).
- Course:** A dropdown menu showing "CSIT".
- Address:** A text area containing "Gongabu, Kathmandu".
- Buttons:** Two buttons, "Submit" and "Clear", are located below the form fields.
- Output:** At the bottom of the window, there is a text area displaying the form data: "Name: Sushank", "Gender: Male", "Hobbies: Music Sports", "Course: CSIT", and "Address: Gongabu, Kathmandu".

17. Write a program to demonstrate different layout manager (null, flow layout, card layout, border layout, grid layout, grid bag layout and group layout).

Program:

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class LayoutManager_17 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Layout Manager Demo");
        frame.setSize(600, 450);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTabbedPane tabs = new JTabbedPane();
        tabs.add("Null", createNullLayoutPanel());
        tabs.add("Flow", createFlowLayoutPanel());
        tabs.add("Card", createCardLayoutPanel());
        tabs.add("Border", createBorderLayoutPanel());
        tabs.add("Grid", createGridLayoutPanel());
        tabs.add("GridBag", createGridBagLayoutPanel());
        tabs.add("Group", createGroupLayoutPanel());

        frame.add(tabs);
        frame.setVisible(true);
    }

    static JPanel createNullLayoutPanel() {
        JPanel panel = new JPanel();
        panel.setLayout(null);

        JLabel label = new JLabel("Null Layout");
        label.setBounds(40, 40, 120, 25);
        panel.add(label);

        JButton button = new JButton("Button");
        button.setBounds(40, 80, 100, 30);
        panel.add(button);

        JTextField textField = new JTextField("Fixed position");
        textField.setBounds(40, 130, 160, 30);
        panel.add(textField);

        return panel;
    }

    static JPanel createFlowLayoutPanel() {
        JPanel panel = new JPanel(new FlowLayout());

        panel.add(new JButton("One"));
        panel.add(new JButton("Two"));
        panel.add(new JButton("Three"));
        panel.add(new JButton("Four"));

        return panel;
    }

    static JPanel createCardLayoutPanel() {
        JPanel mainPanel = new JPanel(new BorderLayout());
        CardLayout cardLayout = new CardLayout();
        JPanel cardPanel = new JPanel(cardLayout);
```

```

    JPanel firstCard = new JPanel();
    firstCard.add(new JLabel("This is first card"));

    JPanel secondCard = new JPanel();
    secondCard.add(new JLabel("This is second card"));

    cardPanel.add(firstCard, "First");
    cardPanel.add(secondCard, "Second");

    JButton nextButton = new JButton("Next Card");
    nextButton.addActionListener(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            cardLayout.next(cardPanel);
        }
    });

    mainPanel.add(cardPanel, BorderLayout.CENTER);
    mainPanel.add(nextButton, BorderLayout.SOUTH);

    return mainPanel;
}

static JPanel createBorderLayoutPanel() {
    JPanel panel = new JPanel(new BorderLayout());

    panel.add(new JButton("North"), BorderLayout.NORTH);
    panel.add(new JButton("South"), BorderLayout.SOUTH);
    panel.add(new JButton("East"), BorderLayout.EAST);
    panel.add(new JButton("West"), BorderLayout.WEST);
    panel.add(new JButton("Center"), BorderLayout.CENTER);

    return panel;
}

static JPanel createGridLayoutPanel() {
    JPanel panel = new JPanel(new GridLayout(2, 3));

    panel.add(new JButton("1"));
    panel.add(new JButton("2"));
    panel.add(new JButton("3"));
    panel.add(new JButton("4"));
    panel.add(new JButton("5"));
    panel.add(new JButton("6"));

    return panel;
}

static JPanel createGridBagLayoutPanel() {
    JPanel panel = new JPanel(new GridBagLayout());
    GridBagConstraints gbc = new GridBagConstraints();

    gbc.gridx = 0;
    gbc.gridy = 0;
    panel.add(new JLabel("Name:"), gbc);

    gbc.gridx = 1;
    gbc.gridy = 0;
    panel.add(new JTextField(12), gbc);

    gbc.gridx = 0;

```

```

        gbc.gridy = 1;
        panel.add(new JLabel("Class:"), gbc);

        gbc.gridx = 1;
        gbc.gridy = 1;
        panel.add(new JTextField(12), gbc);

        gbc.gridx = 1;
        gbc.gridy = 2;
        panel.add(new JButton("Submit"), gbc);

        return panel;
    }

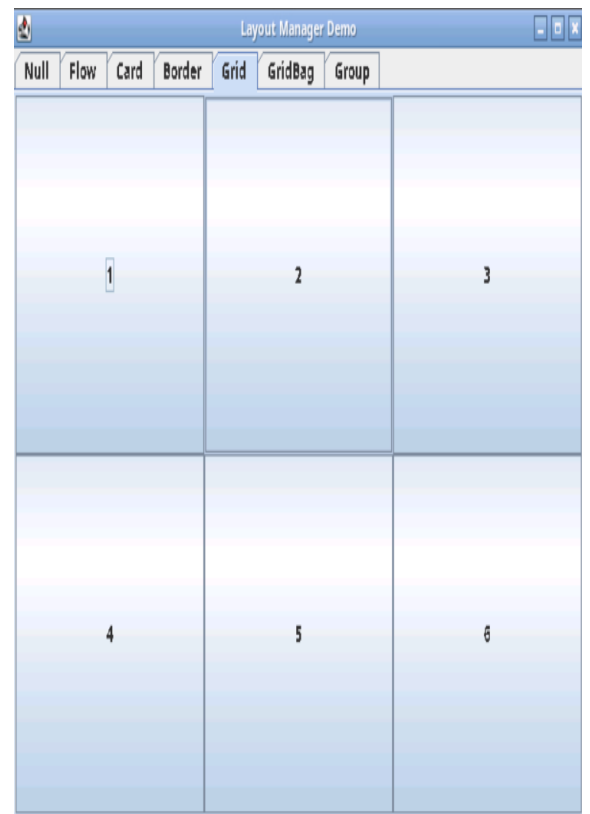
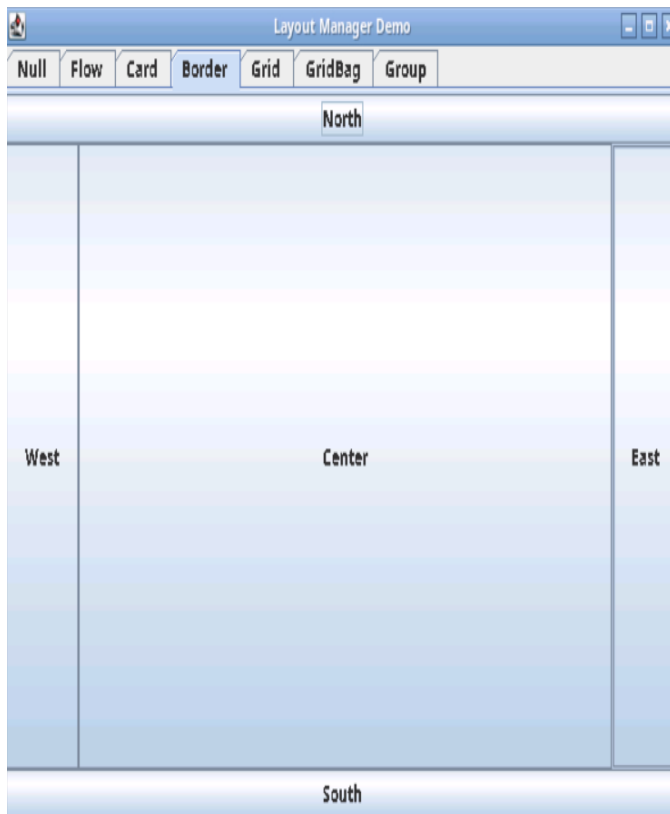
    static JPanel createGroupLayoutPanel() {
        JPanel panel = new JPanel();

        JLabel nameLabel = new JLabel("Name:");
        JTextField nameTextField = new JTextField(12);
        JLabel addressLabel = new JLabel("Address:");
        JTextField addressTextField = new JTextField(12);
        JButton saveButton = new JButton("Save");

        GroupLayout layout = new GroupLayout(panel);
        panel.setLayout(layout);
        return panel;
    }
}

```

Output:



18. Create four button and one text field. If first button is clicked display first button is clicked in text field and same for all other button.

Program:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Buttons_18 extends JFrame implements ActionListener {
    JTextField textField;
    JButton b1, b2, b3, b4;

    public Buttons_18() {
        setTitle("Button Click Demo");
        setSize(400, 200);
        setLayout(new FlowLayout());
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Text Field
        textField = new JTextField(25);
        add(textField);

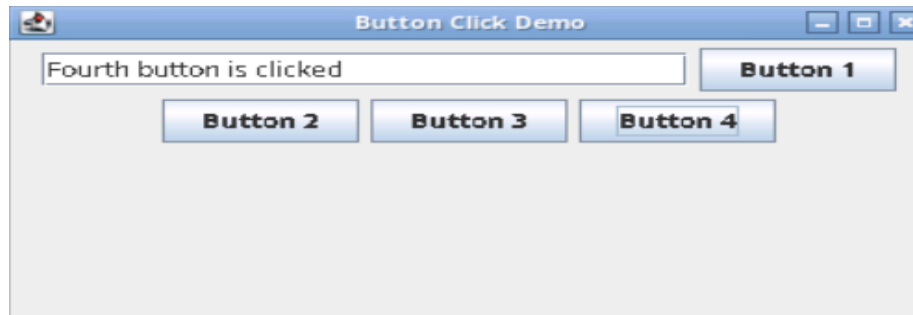
        // Buttons
        b1 = new JButton("Button 1");
        b2 = new JButton("Button 2");
        b3 = new JButton("Button 3");
        b4 = new JButton("Button 4");
        add(b1);
        add(b2);
        add(b3);
        add(b4);

        // Add Action Listener
        b1.addActionListener(this);
        b2.addActionListener(this);
        b3.addActionListener(this);
        b4.addActionListener(this);
        setVisible(true);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == b1) {
            textField.setText("First button is clicked");
        } else if (e.getSource() == b2) {
            textField.setText("Second button is clicked");
        } else if (e.getSource() == b3) {
            textField.setText("Third button is clicked");
        } else if (e.getSource() == b4) {
            textField.setText("Fourth button is clicked");
        }
    }

    public static void main(String[] args) {
        new Buttons_18();
    }
}
```

Output:



19. How are JTable and JComboBox created? Show with Program

Program:

```
import javax.swing.*;
import java.awt.*;

public class Table_Combo_19 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JTable and JComboBox Example");
        frame.setSize(500, 350);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout());

        JLabel courseLabel = new JLabel("Select Course:");
        String[] courses = { "BCA", "BBS", "BIM", "CSIT" };
        JComboBox<String> courseComboBox = new JComboBox<>(courses);

        JPanel topPanel = new JPanel();
        topPanel.add(courseLabel);
        topPanel.add(courseComboBox);
        frame.add(topPanel, BorderLayout.NORTH);

        String[] columns = { "Roll No", "Name", "Address" };
        String[][] data = {
            { "1", "Sushank", "Kathmandu" },
            { "2", "Sita", "Pokhara" },
            { "3", "Hari", "Lalitpur" },
            { "4", "Gita", "Bhaktapur" }
        };

        JTable table = new JTable(data, columns);
        JScrollPane scrollPane = new JScrollPane(table);
        frame.add(scrollPane, BorderLayout.CENTER);

        JLabel selectedLabel = new JLabel("Selected Course: BCA");
        frame.add(selectedLabel, BorderLayout.SOUTH);

        courseComboBox.addActionListener(e -> {
            selectedLabel.setText("Selected Course: " +
courseComboBox.getSelectedItem());
        });

        frame.setVisible(true);
    }
}
```

Output:

JTable and JComboBox Example

Select Course: CSIT

Roll No	Name	Address
1	Sushank	Kathmandu
2	Sita	Pokhara
3	Hari	Lalitpur
4	Gita	Bhaktapur

Selected Course: CSIT

20. Write a program in swing to display menu (with menu item, radio button menu item checkbox menu item and mnemonics), dialogue box, file chooser, color chooser and trees

Program:

```
import javax.swing.*;
import java.awt.*;
import javax.swing.tree.DefaultMutableTreeNode;

public class Menu_20 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Menu Example");
        frame.setSize(550, 450);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setLayout(new BorderLayout());

        JLabel messageLabel = new JLabel("Use menu options to see the result.");
        frame.add(messageLabel, BorderLayout.SOUTH);

        JMenuBar menuBar = new JMenuBar();

        JMenu fileMenu = new JMenu("File");
        fileMenu.setMnemonic('F');

        JMenuItem openItem = new JMenuItem("Open File");
        openItem.setMnemonic('O');
        openItem.setAccelerator(KeyStroke.getKeyStroke("ctrl O"));

        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.setMnemonic('X');

        fileMenu.add(openItem);
        fileMenu.add(exitItem);

        JMenu optionMenu = new JMenu("Options");
        optionMenu.setMnemonic('O');

        JRadioButtonMenuItem smallItem = new JRadioButtonMenuItem("Small Text");
        JRadioButtonMenuItem largeItem = new JRadioButtonMenuItem("Large Text");
        smallItem.setMnemonic('S');
        largeItem.setMnemonic('L');

        ButtonGroup sizeGroup = new ButtonGroup();
        sizeGroup.add(smallItem);
        sizeGroup.add(largeItem);
    }
}
```

```

JCheckBoxMenuItem boldItem = new JCheckBoxMenuItem("Bold");
boldItem.setMnemonic('B');

optionMenu.add(smallItem);
optionMenu.add(largeItem);
optionMenu.add(boldItem);

JMenu toolsMenu = new JMenu("Tools");
toolsMenu.setMnemonic('T');

JMenuItem dialogItem = new JMenuItem("Dialog Box");
dialogItem.setMnemonic('D');

JMenuItem colorItem = new JMenuItem("Color Chooser");
colorItem.setMnemonic('C');

toolsMenu.add(dialogItem);
toolsMenu.add(colorItem);

menuBar.add(fileMenu);
menuBar.add(optionMenu);
menuBar.add(toolsMenu);

frame.setJMenuBar(menuBar);

DefaultMutableTreeNode root = new
DefaultMutableTreeNode("Computer");
DefaultMutableTreeNode documents = new
DefaultMutableTreeNode("Documents");
DefaultMutableTreeNode pictures = new
DefaultMutableTreeNode("Pictures");
DefaultMutableTreeNode music = new
DefaultMutableTreeNode("Music");

documents.add(new DefaultMutableTreeNode("Java Notes"));
documents.add(new DefaultMutableTreeNode("Lab Report"));
pictures.add(new DefaultMutableTreeNode("Photo"));
music.add(new DefaultMutableTreeNode("Song"));

root.add(documents);
root.add(pictures);
root.add(music);

JTree tree = new JTree(root);
JScrollPane treeScrollPane = new JScrollPane(tree);

JPanel centerPanel = new JPanel(new BorderLayout());
centerPanel.add(new JLabel("JTree Example:"),
BorderLayout.NORTH);
centerPanel.add(treeScrollPane, BorderLayout.CENTER);
frame.add(centerPanel, BorderLayout.CENTER);

openItem.addActionListener(e -> {
    JFileChooser fileChooser = new JFileChooser();
    int result = fileChooser.showOpenDialog(frame);

    if (result == JFileChooser.APPROVE_OPTION) {
        messageLabel.setText("Selected file: " +
fileChooser.getSelectedFile().getName());
    } else {
        messageLabel.setText("No file selected.");
    }
});

```

```

    }
});

exitItem.addActionListener(e -> {
    System.exit(0);
});

smallItem.addActionListener(e -> {
    messageLabel.setText("Small text menu item selected.");
});

largeItem.addActionListener(e -> {
    messageLabel.setText("Large text menu item selected.");
});

boldItem.addActionListener(e -> {
    if (boldItem.isSelected()) {
        messageLabel.setText("Checkbox menu item selected.");
    } else {
        messageLabel.setText("Checkbox menu item unselected.");
    }
});

dialogItem.addActionListener(e -> {
    JOptionPane.showMessageDialog(frame, "This is a dialog
box.");
    messageLabel.setText("Dialog box displayed.");
});

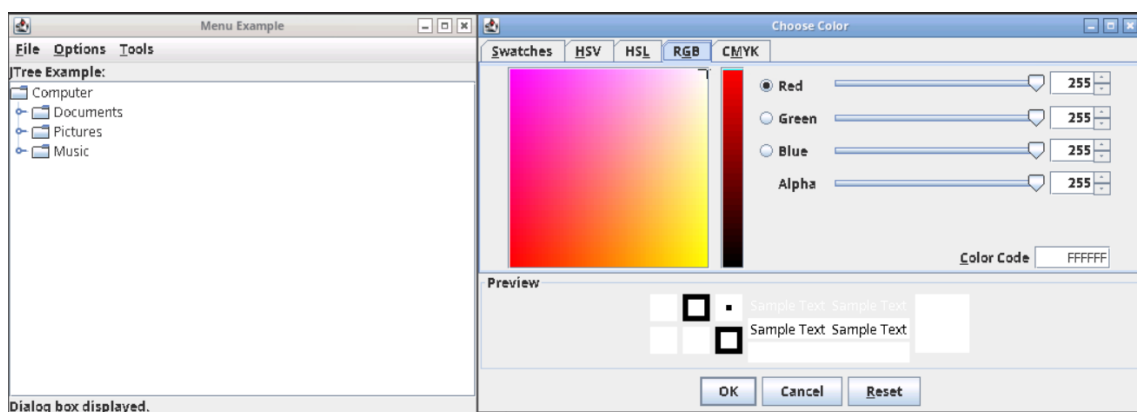
colorItem.addActionListener(e -> {
    Color color = JColorChooser.showDialog(frame, "Choose Color",
Color.WHITE);

    if (color != null) {
        centerPanel.setBackground(color);
        messageLabel.setText("Color selected.");
    } else {
        messageLabel.setText("No color selected.");
    }
});

frame.setVisible(true);
}
}

```

Output:



Chapter 3:

21.WAP to handle action event using radio button and normal button.

Program:

```
import javax.swing.*;

public class ButtonEvent_21 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Button Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JRadioButton maleRadioButton = new JRadioButton("Male");
        maleRadioButton.setBounds(80, 40, 80, 30);
        frame.add(maleRadioButton);

        JRadioButton femaleRadioButton = new JRadioButton("Female");
        femaleRadioButton.setBounds(180, 40, 100, 30);
        frame.add(femaleRadioButton);

        ButtonGroup genderGroup = new ButtonGroup();
        genderGroup.add(maleRadioButton);
        genderGroup.add(femaleRadioButton);

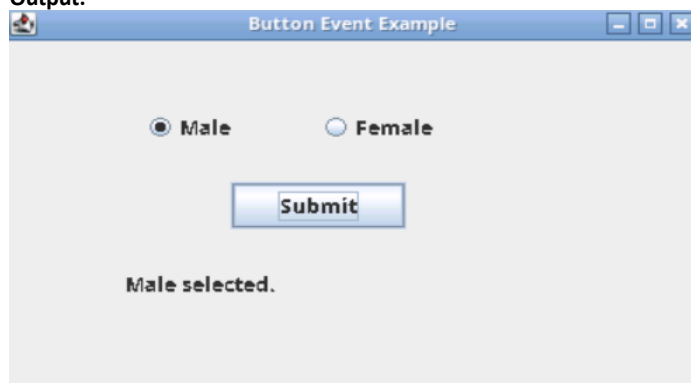
        JButton button = new JButton("Submit");
        button.setBounds(130, 90, 100, 30);
        frame.add(button);

        JLabel resultLabel = new JLabel("Select radio button and click  
submit.");
        resultLabel.setBounds(70, 140, 280, 30);
        frame.add(resultLabel);

        button.addActionListener(e -> {
            if (maleRadioButton.isSelected()) {
                resultLabel.setText("Male selected.");
            } else if (femaleRadioButton.isSelected()) {
                resultLabel.setText("Female selected.");
            } else {
                resultLabel.setText("No radio button selected.");
            }
        });

        frame.setVisible(true);
    }
}
```

Output:



22.WAP to handle item event using checkbox

Program:

```
import javax.swing.*;

public class CheckBoxEvent_22 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("CheckBox Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JCheckBox teaCheckBox = new JCheckBox("Tea");
        teaCheckBox.setBounds(80, 40, 80, 30);
        frame.add(teaCheckBox);

        JCheckBox coffeeCheckBox = new JCheckBox("Coffee");
        coffeeCheckBox.setBounds(180, 40, 100, 30);
        frame.add(coffeeCheckBox);

        JLabel resultLabel = new JLabel("Selected: None");
        resultLabel.setBounds(80, 100, 250, 30);
        frame.add(resultLabel);

        teaCheckBox.addItemListener(e -> {
            String selected = "";

            if (teaCheckBox.isSelected()) {
                selected = selected + "Tea ";
            }
            if (coffeeCheckBox.isSelected()) {
                selected = selected + "Coffee ";
            }
            if (selected.equals("")) {
                selected = "None";
            }

            resultLabel.setText("Selected: " + selected);
        });

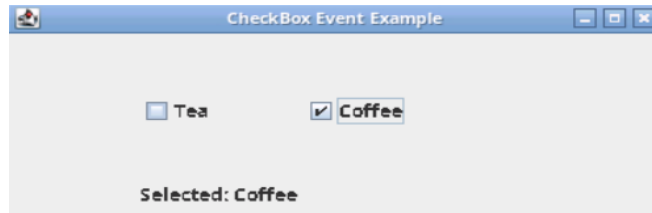
        coffeeCheckBox.addItemListener(e -> {
            String selected = "";

            if (teaCheckBox.isSelected()) {
                selected = selected + "Tea ";
            }
            if (coffeeCheckBox.isSelected()) {
                selected = selected + "Coffee ";
            }
            if (selected.equals("")) {
                selected = "None";
            }

            resultLabel.setText("Selected: " + selected);
        });

        frame.setVisible(true);
    }
}
```

Output:



23. WAP to handle Mouse Event.

Program:

```
import java.awt.event.MouseAdapter;
import java.awt.event.MouseEvent;
import javax.swing.*.*;

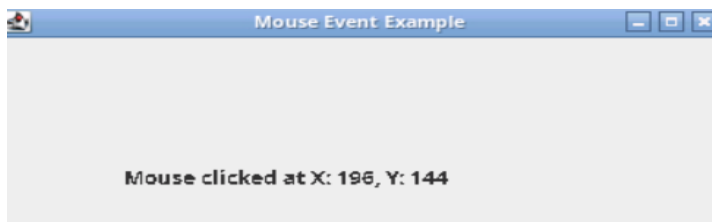
public class MouseEvent_23 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Mouse Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel resultLabel = new JLabel("Click anywhere inside the
window.");
        resultLabel.setBounds(70, 80, 280, 30);
        frame.add(resultLabel);

        frame.addMouseListener(new MouseAdapter() {
            public void mouseClicked(MouseEvent e) {
                resultLabel.setText("Mouse clicked at X: " + e.getX() +
", Y: " + e.getY());
            }
        });

        frame.setVisible(true);
    }
}
```

Output:



24. WAP to handle Key Event

Program:

```
import java.awt.event.KeyAdapter;
import java.awt.event.KeyEvent;
import javax.swing.*.*;

public class KeyEvent_24 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Key Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
```

```

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextField textField = new JTextField();
        textField.setBounds(80, 50, 220, 30);
        frame.add(textField);

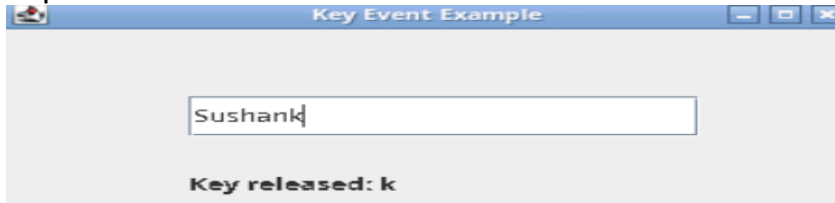
        JLabel resultLabel = new JLabel("Type something in the text
field.");
        resultLabel.setBounds(80, 100, 250, 30);
        frame.add(resultLabel);

        textField.addKeyListener(new KeyAdapter() {
            public void keyReleased(KeyEvent e) {
                resultLabel.setText("Key released: " + e.getKeyChar());
            }
        });

        frame.setVisible(true);
    }
}

```

Output:



25. WAP to handle List Selection Event.

Program:

```

import javax.swing.*;

public class ListSelectionEvent_25 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("List Selection Event Example");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        String[] courses = {"BCA", "BBS", "BIM", "CSIT"};
        JList<String> courseList = new JList<>(courses);

        JScrollPane scrollPane = new JScrollPane(courseList);
        scrollPane.setBounds(80, 40, 120, 90);
        frame.add(scrollPane);

        JLabel resultLabel = new JLabel("Select a course.");
        resultLabel.setBounds(80, 150, 250, 30);
        frame.add(resultLabel);

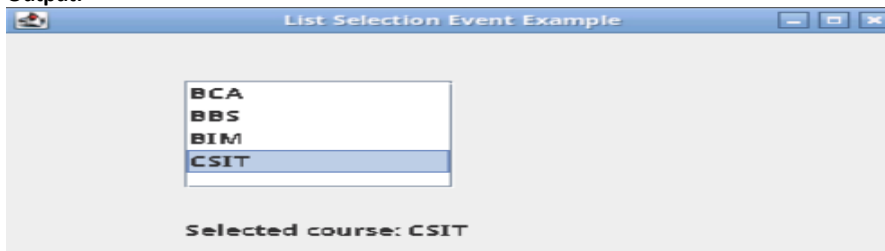
        courseList.addListSelectionListener(e -> {
            if (!e.getValueIsAdjusting()) {
                resultLabel.setText("Selected course: " +
courseList.getSelectedValue());
            }
        });

        frame.setVisible(true);
    }
}

```

```
}
```

Output:



26. WAP to handle Focus Event.

Program:

```
import java.awt.event.FocusAdapter;
import java.awt.event.FocusEvent;
import javax.swing.*.*;

public class FocusEvent_26 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Focus Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JTextField nameTextField = new JTextField();
        nameTextField.setBounds(80, 50, 220, 30);
        frame.add(nameTextField);

        JTextField addressTextField = new JTextField();
        addressTextField.setBounds(80, 90, 220, 30);
        frame.add(addressTextField);

        JLabel resultLabel = new JLabel("Click inside a text field.");
        resultLabel.setBounds(80, 140, 250, 30);
        frame.add(resultLabel);

        nameTextField.addFocusListener(new FocusAdapter() {
            public void focusGained(FocusEvent e) {
                resultLabel.setText("Name field got focus.");
            }

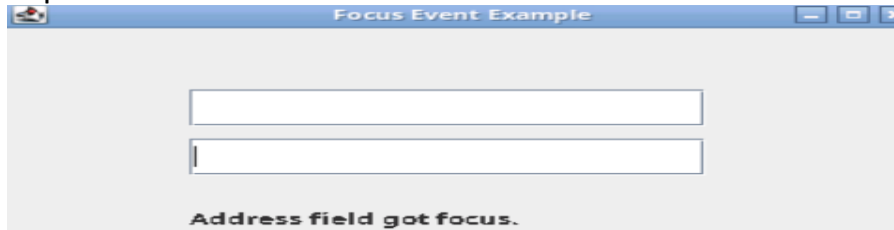
            public void focusLost(FocusEvent e) {
                resultLabel.setText("Name field lost focus.");
            }
        });

        addressTextField.addFocusListener(new FocusAdapter() {
            public void focusGained(FocusEvent e) {
                resultLabel.setText("Address field got focus.");
            }

            public void focusLost(FocusEvent e) {
                resultLabel.setText("Address field lost focus.");
            }
        });

        frame.setVisible(true);
    }
}
```

Output:



27. WAP to handle Window Event.

Program:

```
import java.awt.event.WindowAdapter;
import java.awt.event.WindowEvent;
import javax.swing.*;

public class WindowEvent_27 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Window Event Example");
        frame.setSize(400, 250);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel resultLabel = new JLabel("Open, minimize, restore, or
close the window.");
        resultLabel.setBounds(50, 80, 320, 30);
        frame.add(resultLabel);

        frame.addWindowListener(new WindowAdapter() {
            public void windowOpened(WindowEvent e) {
                resultLabel.setText("Window opened.");
            }

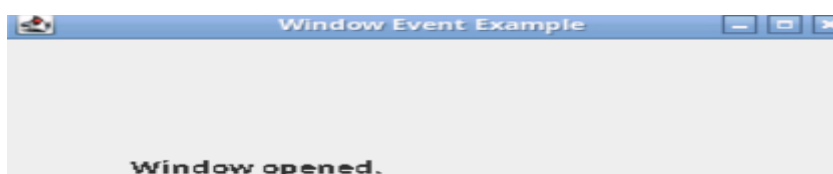
            public void windowIconified(WindowEvent e) {
                System.out.println("Window minimized.");
            }

            public void windowDeiconified(WindowEvent e) {
                resultLabel.setText("Window restored.");
            }

            public void windowClosing(WindowEvent e) {
                System.out.println("Window closing.");
            }
        });

        frame.setVisible(true);
    }
}
```

Output:



28. Write a program to find the sum and difference of two number. Use two text field to take input and another one text field to display result. Display sum of two number if mouse is clicked and display difference if mouse is released.

Program:

```
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class Add_Sub_28 {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Add and Subtract");
        frame.setSize(400, 300);
        frame.setLayout(null);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JLabel firstLabel = new JLabel("First Number:");
        firstLabel.setBounds(50, 40, 100, 25);
        frame.add(firstLabel);

        JTextField firstTextField = new JTextField();
        firstTextField.setBounds(170, 40, 150, 25);
        frame.add(firstTextField);

        JLabel secondLabel = new JLabel("Second Number:");
        secondLabel.setBounds(50, 80, 120, 25);
        frame.add(secondLabel);

        JTextField secondTextField = new JTextField();
        secondTextField.setBounds(170, 80, 150, 25);
        frame.add(secondTextField);

        JButton addButton = new JButton("Add");
        addButton.setBounds(80, 130, 100, 30);
        frame.add(addButton);

        JButton subtractButton = new JButton("Subtract");
        subtractButton.setBounds(200, 130, 100, 30);
        frame.add(subtractButton);

        JLabel resultLabel = new JLabel("Result will be displayed here.");
        resultLabel.setBounds(80, 185, 250, 30);
        frame.add(resultLabel);

        addButton.addActionListener(e -> {
            int firstNumber = Integer.parseInt(firstTextField.getText());
            int secondNumber = Integer.parseInt(secondTextField.getText());
            int sum = firstNumber + secondNumber;

            resultLabel.setText("Sum: " + sum);
        });
    }
}
```

```

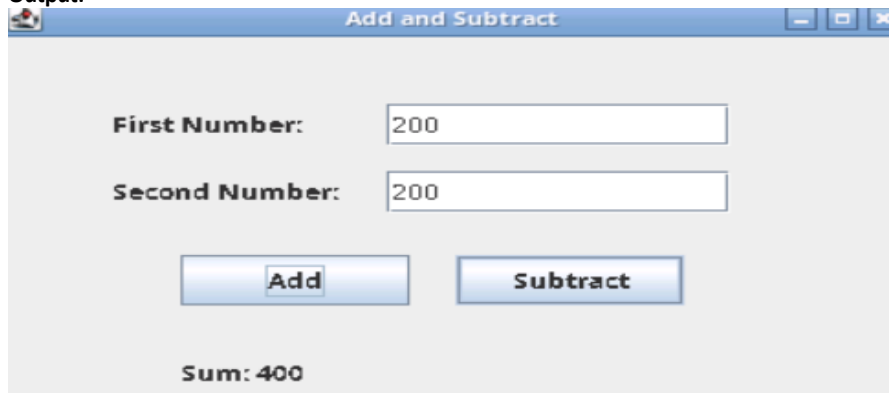
        subtractButton.addActionListener(e -> {
            int firstNumber = Integer.parseInt(firstTextField.getText());
            int secondNumber = Integer.parseInt(secondTextField.getText());
            int difference = firstNumber - secondNumber;

            resultLabel.setText("Difference: " + difference);
        });

        frame.setVisible(true);
    }
}

```

Output:



29. WAP to perform CRUD Operation

Program:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class Crud_29 {
    public static void main(String[] args) {

        try {
            Class.forName("org.sqlite.JDBC");
            Connection connection =
DriverManager.getConnection("jdbc:sqlite:database/student.db");

            String createTableSQL = "CREATE TABLE IF NOT EXISTS students (id INT PRIMARY
KEY, name VARCHAR(50))";
            Statement statement = connection.createStatement();
            statement.execute(createTableSQL);

            // Create
            String insertSQL = "INSERT INTO students (id, name) VALUES (1, 'Sushank')";
            statement.executeUpdate(insertSQL);
            System.out.println("User inserted successfully");

            // Read

```

```

        String selectSQL = "SELECT * FROM students";
        ResultSet resultSet = statement.executeQuery(selectSQL);
        while (resultSet.next()) {
            System.out.println("ID: " + resultSet.getInt("id") +
                               ", Name: " + resultSet.getString("name"));
        }

        // Update
        String updateSQL = "UPDATE students SET name = 'Sushank' WHERE id = 1";
        statement.executeUpdate(updateSQL);
        System.out.println("User updated successfully");

        // Delete
        String deleteSQL = "DELETE FROM students WHERE id = 1";
        statement.executeUpdate(deleteSQL);
        System.out.println("User deleted successfully");

    } catch (SQLException | ClassNotFoundException e) {
        System.out.println(e.getMessage());
    }
}
}

```

Output:

```

User inserted successfully
ID: 1, Name: Sushank
User updated successfully
User deleted successfully
> vscode → .../Test/Sushank/Java/Lab16-31 $ █

```

30.WAP to perform CRUD Operation using Prepared Statement.

Program:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class CrudPrepared_30 {
    public static void main(String[] args) {
        try {
            Class.forName("org.sqlite.JDBC");
            Connection connection =
DriverManager.getConnection("jdbc:sqlite:database/employee.db");

            String createTableSQL = "CREATE TABLE IF NOT EXISTS employees (id INT PRIMARY
KEY, name VARCHAR(50), salary INT)";
            Statement statement = connection.createStatement();

```

```

statement.execute(createTableSQL);

// Create operation
String insertSQL = "INSERT INTO employees (id, name, salary) VALUES (?, ?,
?));

PreparedStatement insertStatement = connection.prepareStatement(insertSQL);
insertStatement.setInt(1, 1);
insertStatement.setString(2, "Sushank Lamsal");
insertStatement.setInt(3, 25000);
insertStatement.executeUpdate();
System.out.println("Employee inserted successfully");

// Read operation
String selectSQL = "SELECT * FROM employees";
PreparedStatement selectStatement = connection.prepareStatement(selectSQL);
ResultSet resultSet = selectStatement.executeQuery();

System.out.println("Employee Records:");
while (resultSet.next()) {
    System.out.println("ID: " + resultSet.getInt("id")
        + ", Name: " + resultSet.getString("name")
        + ", Salary: " + resultSet.getInt("salary"));
}

// Update operation
String updateSQL = "UPDATE employees SET name = ?, salary = ? WHERE id = ?";
PreparedStatement updateStatement = connection.prepareStatement(updateSQL);
updateStatement.setString(1, "Hari");
updateStatement.setInt(2, 30000);
updateStatement.setInt(3, 1);
updateStatement.executeUpdate();
System.out.println("Employee updated successfully");

// Delete operation
String deleteSQL = "DELETE FROM employees WHERE id = ?";
PreparedStatement deleteStatement = connection.prepareStatement(deleteSQL);
deleteStatement.setInt(1, 1);
deleteStatement.executeUpdate();
System.out.println("Employee deleted successfully");

resultSet.close();
statement.close();
insertStatement.close();
selectStatement.close();
updateStatement.close();
deleteStatement.close();

```

```

        connection.close();
    } catch (SQLException | ClassNotFoundException e) {
        System.out.println(e.getMessage());
    }
}
}
}

```

Output:

```

Employee inserted successfully
Employee Records:
ID: 1, Name: Sushank Lamsal, Salary: 25000
Employee updated successfully
Employee deleted successfully
vscode → .../Test/Sushank/Java/Lab16-31 $

```

31.Demonstrate the use of row set and updatable and scrollable result set.

Program:

```

import java.sql.*;
import javax.sql.rowset.CachedRowSet;
import javax.sql.rowset.RowSetProvider;

public class rowset_31 {
    public static void main(String[] args) {
        try {
            Class.forName("org.sqlite.JDBC");
            Connection connection =
DriverManager.getConnection("jdbc:sqlite:database/student.db");
            Statement statement = connection.createStatement();

            ResultSet resultSet = statement.executeQuery("SELECT id, name FROM students");

            CachedRowSet rowSet = RowSetProvider.newFactory().createCachedRowSet();
            rowSet.populate(resultSet);

            if (!rowSet.first()) {
                System.out.println("No records found in student.db");
                resultSet.close();
                statement.close();
                connection.close();
                return;
            }

            System.out.println("All records using RowSet:");
            rowSet.beforeFirst();
            while (rowSet.next()) {
                System.out.println(rowSet.getInt("id") + " "
                    + rowSet.getString("name"));
            }

            System.out.println("\nScrollable RowSet Example:");

```

```

        rowSet.first();
        System.out.println("First record: " + rowSet.getString("name"));

        rowSet.last();
        System.out.println("Last record: " + rowSet.getString("name"));

        rowSet.first();
        rowSet.updateString("name", "Sushank Lamsal");
        rowSet.updateRow();
        System.out.println("\nFirst row updated in RowSet.");

        PreparedStatement updateStatement = connection
            .prepareStatement("UPDATE students SET name = ? WHERE id = ?");
        updateStatement.setString(1, rowSet.getString("name"));
        updateStatement.setInt(2, rowSet.getInt("id"));
        updateStatement.executeUpdate();

        System.out.println("\nRecords after update:");
        ResultSet updatedResultSet = statement.executeQuery("SELECT id, name FROM
students");
        while (updatedResultSet.next()) {
            System.out.println(updatedResultSet.getInt("id") + " "
                + updatedResultSet.getString("name"));
        }

        updatedResultSet.close();
        resultSet.close();
        updateStatement.close();
        statement.close();
        connection.close();
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}
}

```

Output:

```

All records using RowSet:
1 Sushank
2 Harry
3 Milo

Scrollable RowSet Example:
First record: Sushank
Last record: Milo

First row updated in RowSet.

Records after update:
1 Sushank Lamsal
2 Harry
3 Milo
vscode → .../Test/Sushank/Java/Lab16-31 $

```

32. Create one complete form using swing

(id username password, gender (button), course (check box), country (combo box), submit and reset button) validate the form

(check for emptiness, password and should be equal) and insert the data into database using prepared statement.

After this create a login form with fields username, password and submit button.

If username and password matched with database record then display all the record of table using rowset or scrollable or updatable result set.

Also perform following operation:

- Update the name of student to "sanu"
- course to "BCA"
- whose id is 3.

Program:

DBConnection.java

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class DBConnection {
    public static Connection getConnection() throws Exception {
        Class.forName("org.sqlite.JDBC");

        Connection connection = DriverManager.getConnection("jdbc:sqlite:student.db");
        createTable(connection);
        return connection;
    }

    static void createTable(Connection connection) throws SQLException {
        String sql = "CREATE TABLE IF NOT EXISTS students ("
            + "id INTEGER PRIMARY KEY, "
            + "username TEXT NOT NULL, "
            + "password TEXT NOT NULL, "
            + "gender TEXT NOT NULL, "
            + "course TEXT NOT NULL, "
            + "country TEXT NOT NULL, "
            + "address TEXT NOT NULL"
            + ")";

        Statement statement = connection.createStatement();
        statement.execute(sql);
        statement.close();
    }
}
```

LoginForm.java

```

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import javax.swing.*.*;

public class LoginForm extends JFrame {
    JTextField usernameTextField;
    JPasswordField passwordField;

    public LoginForm() {
        setTitle("Login Form");
        setSize(400, 250);
        setLayout(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

        JLabel usernameLabel = new JLabel("Username:");
        usernameLabel.setBounds(50, 50, 100, 25);
        add(usernameLabel);

        usernameTextField = new JTextField();
        usernameTextField.setBounds(160, 50, 170, 25);
        add(usernameTextField);

        JLabel passwordLabel = new JLabel("Password:");
        passwordLabel.setBounds(50, 90, 100, 25);
        add(passwordLabel);

        passwordField = new JPasswordField();
        passwordField.setBounds(160, 90, 170, 25);
        add(passwordField);

        JButton loginButton = new JButton("Login");
        loginButton.setBounds(150, 140, 100, 30);
        add(loginButton);

        loginButton.addActionListener(e -> login());
    }

    void login() {
        String username = usernameTextField.getText();
        String password = new String(passwordField.getPassword());

        if (username.equals("") || password.equals("")) {
            JOptionPane.showMessageDialog(this, "Please enter username and password.");
            return;
        }

        try {
            Connection connection = DBConnection.getConnection();
            String sql = "SELECT * FROM students WHERE username = ? AND password = ?";
            PreparedStatement statement = connection.prepareStatement(sql);
            statement.setString(1, username);
            statement.setString(2, password);

```

```

        ResultSet resultSet = statement.executeQuery();

        if (resultSet.next()) {
            JOptionPane.showMessageDialog(this, "Login successful.");
            new DisplayData().setVisible(true);
            dispose();
        } else {
            JOptionPane.showMessageDialog(this, "Invalid username or password.");
        }

        resultSet.close();
        statement.close();
        connection.close();
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this, "Error: " + e.getMessage());
    }
}
}
}

```

Output:

ID	Username	Password	Gender	Course	Country	Address
1	Sushank	hello	Male	CSIT	Nepal	Gongabu,KTM
2	Hari devkota	hi123	Male	BCA	Nepal	Bhaktapur
3	sanu	sanu123	Female	BCA	Nepal	Lalitpur

ID	Username	Password	Gender	Course	Country	Address
1	Sushank	hello	Male	CSIT	Nepal	Gongabu,KTM
2	Hari devkota	hi123	Male	BCA	Nepal	Bhaktapur
3	sanu	sanu123	Female	CSIT	Nepal	Lalitpur

33. Write a Java program to implement TCP.

Program:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class TCP {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Use: java TCP server");
            System.out.println("Or : java TCP client");
            return;
        }

        if (args[0].equalsIgnoreCase("server")) {
            startServer();
        } else if (args[0].equalsIgnoreCase("client")) {
            startClient();
        } else {
            System.out.println("Invalid option. Use server or client.");
        }
    }

    static void startServer() {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("TCP Server started. Waiting for client...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected.");

            BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

            String message = input.readLine();
            System.out.println("Client says: " + message);

            output.println("Hello client, message received.");

            socket.close();
            serverSocket.close();
        } catch (Exception e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }

    static void startClient() {
        try {
            Socket socket = new Socket("localhost", 5000);
```

```

        BufferedReader keyboard = new BufferedReader(new
InputStreamReader(System.in));
        BufferedReader input = new BufferedReader(new
InputStreamReader(socket.getInputStream()));
        PrintWriter output = new PrintWriter(socket.getOutputStream(), true);

        System.out.print("Enter message for server: ");
        String message = keyboard.readLine();

        output.println(message);

        String reply = input.readLine();
        System.out.println("Server says: " + reply);

        socket.close();
    } catch (Exception e) {
        System.out.println("Client error: " + e.getMessage());
    }
}
}

```

Output:

```

● vscode →.../Sushank/Java/Lab33-34/src $ javac TCP.java
● vscode →.../Sushank/Java/Lab33-34/src $ java TCP server
TCP Server started. Waiting for client...
Client connected.
Client says: Hey , How are Sushank ?

```

```

● vscode →.../Sushank/Java/Lab33-34/src $ java TCP client
Enter message for server: Hey , How are Sushank ?
Server says: Hello client, message received.

```

34. Write a Java program to implement UDP.

Program:

```

import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.util.Scanner;

public class UDP {
    public static void main(String[] args) {
        if (args.length == 0) {
            System.out.println("Use: java UDP server");
            System.out.println("Or : java UDP client");
        }
    }
}

```

```

        return;
    }

    if (args[0].equalsIgnoreCase("server")) {
        startServer();
    } else if (args[0].equalsIgnoreCase("client")) {
        startClient();
    } else {
        System.out.println("Invalid option. Use server or client.");
    }
}

static void startServer() {
    try {
        DatagramSocket socket = new DatagramSocket(6000);
        byte[] receiveData = new byte[1024];

        System.out.println("UDP Server started. Waiting for message...");

        DatagramPacket receivePacket = new DatagramPacket(receiveData,
receiveData.length);
        socket.receive(receivePacket);

        String message = new String(receivePacket.getData(), 0,
receivePacket.getLength());
        System.out.println("Client says: " + message);

        String reply = "Hello client, message received.";
        byte[] sendData = reply.getBytes();

        InetAddress clientAddress = receivePacket.getAddress();
        int clientPort = receivePacket.getPort();

        DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length,
clientAddress, clientPort);
        socket.send(sendPacket);

        socket.close();
    } catch (Exception e) {
        System.out.println("Server error: " + e.getMessage());
    }
}

static void startClient() {
    try {
        DatagramSocket socket = new DatagramSocket();

```

```

Scanner scanner = new Scanner(System.in);

System.out.print("Enter message for server: ");
String message = scanner.nextLine();

byte[] sendData = message.getBytes();
InetAddress serverAddress = InetAddress.getByName("localhost");

DatagramPacket sendPacket = new DatagramPacket(sendData, sendData.length,
serverAddress, 6000);
socket.send(sendPacket);

byte[] receiveData = new byte[1024];
DatagramPacket receivePacket = new DatagramPacket(receiveData,
receiveData.length);
socket.receive(receivePacket);

String reply = new String(receivePacket.getData(), 0,
receivePacket.getLength());
System.out.println("Server says: " + reply);

scanner.close();
socket.close();
} catch (Exception e) {
    System.out.println("Client error: " + e.getMessage());
}
}
}

```

Output:

```

• vscode →.../Sushank/Java/Lab33-34/src $ javac UDP.java
• vscode →.../Sushank/Java/Lab33-34/src $ java UDP server
UDP Server started. Waiting for message...
Client says: Hello , What is your name ?

```

```

• vscode →.../Sushank/Java/Lab33-34/src $ java UDP client
Enter message for server: Hello , What is your name ?
Server says: Hello client, message received.

```

35.Demonstrate JavaFX with database implementation.

Program:

```

import javafx.application.Application;
import javafx.stage.*;
import javafx.scene.*;
import javafx.scene.control.*;

```

```

import javafx.scene.layout.*;
import javafx.event.*;
import java.sql.*;
public class HelloFX extends Application {
    @Override
    public void start(Stage stage) {
        Label usernameLabel = new Label("Username:");
        Label passwordLabel = new Label("Password:");
        Label rePasswordLabel = new Label("Re-enter Password:");
        Label genderLabel = new Label("Gender:");
        Label courseLabel = new Label("Course:");
        Label countryLabel = new Label("Country:");
        TextField usernameField = new TextField();
        PasswordField passwordField = new PasswordField();
        PasswordField rePasswordField = new PasswordField();
        RadioButton male = new RadioButton("Male");
        RadioButton female = new RadioButton("Female");
        ToggleGroup tg = new ToggleGroup();
        male.setToggleGroup(tg);
        female.setToggleGroup(tg);
        CheckBox javaBox = new CheckBox("Java");
        CheckBox cppBox = new CheckBox("C++");
        ChoiceBox<String> countryBox = new ChoiceBox<>();
        countryBox.getItems().addAll("Nepal", "India", "Other");
        Button submit = new Button("Submit");
        TextField result = new TextField();
        result.setEditable(false);
        GridPane grid = new GridPane();
        grid.add(usernameLabel, 0, 0);
        grid.add(usernameField, 1, 0);
        grid.add(passwordLabel, 0, 1);
        grid.add(passwordField, 1, 1);
        grid.add(rePasswordLabel, 0, 2);
        grid.add(rePasswordField, 1, 2);
        grid.add(genderLabel, 0, 3);
        grid.add(male, 1, 3);
        grid.add(female, 1, 4);
        grid.add(courseLabel, 0, 5);
        grid.add(javaBox, 1, 5);
        grid.add(cppBox, 1, 6);
        grid.add(countryLabel, 0, 7);
        grid.add(countryBox, 1, 7);
        grid.add(submit, 1, 8);
        grid.add(result, 1, 9);
        submit.setOnAction(e -> {
            String username = usernameField.getText();
            String password = passwordField.getText();
            String rePassword = rePasswordField.getText();
            String gender = male.isSelected() ? "Male"
                : female.isSelected() ? "Female" : "";
            String course = "";
            if (javaBox.isSelected()) course += "Java ";
            if (cppBox.isSelected()) course += "C++ ";
            String country = countryBox.getValue();
            if (username.isEmpty() || password.isEmpty() || rePassword.isEmpty()
                || gender.isEmpty() || country == null) {

```

```

        result.setText("Fill all fields!");
        return;
    }
    if (!password.equals(rePassword)) {
        result.setText("Passwords do not match!");
        return;
    }
    Scene scene = new Scene(grid, 450, 400);
    stage.setScene(scene);
    stage.setTitle("Registration Form");
    stage.show();
}
public static void main(String[] args) {
    launch(args);
}
}

```

Output:

36. Write a java program to demonstrate different JavaFX layouts.

Program:

```

import javafx.application.Application;
import javafx.geometry.Pos;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.stage.Stage;

public class HelloFX extends Application {
    @Override
    public void start(Stage stage) {

        FlowPane flowPane = new FlowPane(10, 10);
        flowPane.getChildren().addAll(
            new Button("Button 1"),
            new Button("Button 2"),
            new Button("Button 3"),
            new Label("FlowPane Layout")
        );
        GridPane gridPane = new GridPane();
        gridPane.setHgap(10);
        gridPane.setVgap(10);
        gridPane.add(new Button("B1"), 0, 0);
        gridPane.add(new Button("B2"), 1, 0);
    }
}

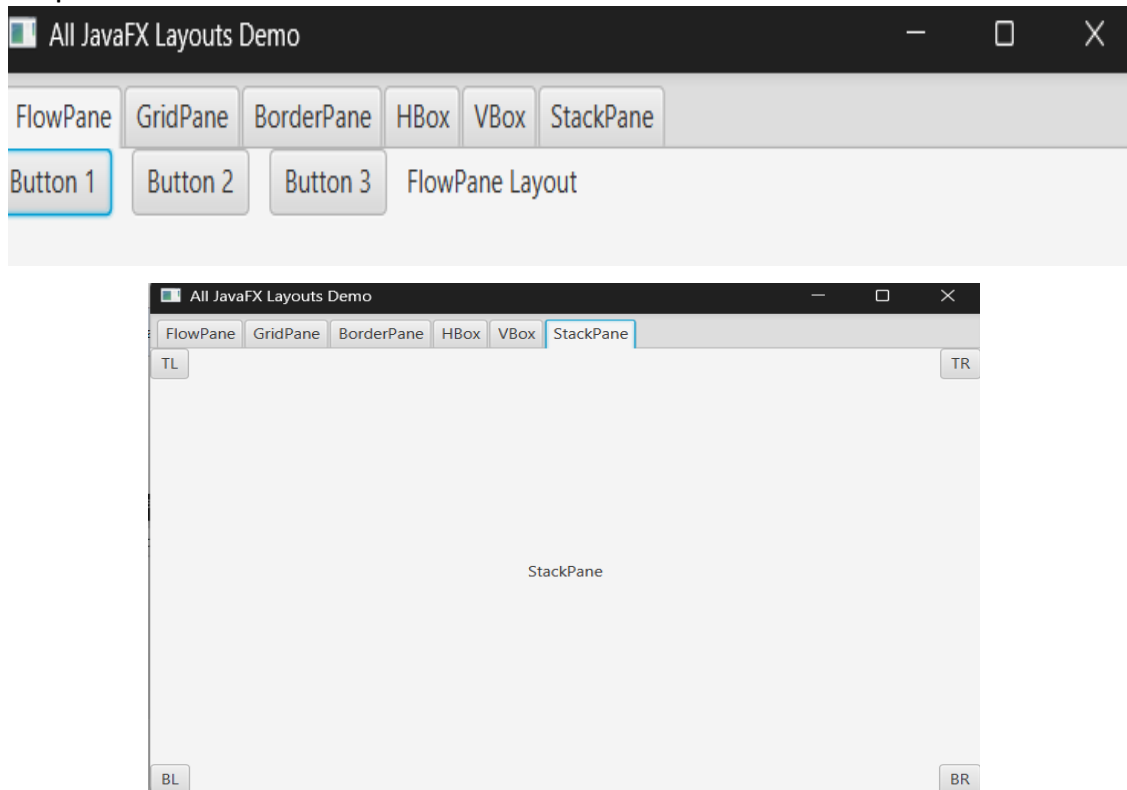
```

```

gridPane.add(new Button("B3"), 0, 1);
gridPane.add(new Label("GridPane Layout"), 1, 1);
BorderPane borderPane = new BorderPane();
borderPane.setTop(new Button("Top"));
borderPane.setBottom(new Button("Bottom"));
borderPane.setLeft(new Button("Left"));
borderPane.setRight(new Button("Right"));
borderPane.setCenter(new Label("BorderPane Center"));
HBox hBox = new HBox(10);
hBox.getChildren().addAll(
    new Button("Button 1"),
    new Button("Button 2"),
    new Button("Button 3"),
    new Label("HBox Layout")
);
VBox vBox = new VBox(10);
vBox.getChildren().addAll(
    new Button("Button 1"),
    new Button("Button 2"),
    new Button("Button 3"),
    new Label("VBox Layout")
);
StackPane stackPane = new StackPane();
Button topLeft = new Button("TL");
Button topRight = new Button("TR");
Button bottomLeft = new Button("BL");
Button bottomRight = new Button("BR");
Label centerLabel = new Label("StackPane");
stackPane.getChildren().addAll(
    topLeft, topRight,
    bottomLeft, bottomRight,
    centerLabel
);
StackPane.setAlignment(topLeft, Pos.TOP_LEFT);
StackPane.setAlignment(topRight, Pos.TOP_RIGHT);
StackPane.setAlignment(bottomLeft, Pos.BOTTOM_LEFT);
StackPane.setAlignment(bottomRight, Pos.BOTTOM_RIGHT);
StackPane.setAlignment(centerLabel, Pos.CENTER);
TabPane tabPane = new TabPane();
tabPane.getTabs().add(new Tab("FlowPane", flowPane));
tabPane.getTabs().add(new Tab("GridPane", gridPane));
tabPane.getTabs().add(new Tab("BorderPane", borderPane));
tabPane.getTabs().add(new Tab("HBox", hBox));
tabPane.getTabs().add(new Tab("VBox", vBox));
tabPane.getTabs().add(new Tab("StackPane", stackPane));
for (Tab tab : tabPane.getTabs()) {
    tab.setClosable(false);
}
Scene scene = new Scene(tabPane, 600, 400);
stage.setTitle("All JavaFX Layouts Demo");
stage.setScene(scene);
stage.show();
}
public static void main(String[] args) {
    launch(args);
}
}

```

Output:



37. Write a program using java server pages (jsp) including pages register, login, dashboard and handle the database for dynamic operations and use session to validate user in dashboard.

Program:

Db.jsp

```
<%@ page import="java.sql.*" %>
<% Connection con = null;
try {
    Class.forName("org.sqlite.JDBC");
    Connection conn = DriverManager.getConnection("jdbc:sqlite:database/student.db");
} catch (Exception e) {
    out.println("DB Error: " + e.getMessage());
}
%>
```

Registration.jsp

```
<%@ page import="java.sql.*" %>
<%@ include file="db.jsp" %>
<%
String msg = "";
String msgClass =
""
if ("POST".equalsIgnoreCase(request.getMethod())) { String
    name = request.getParameter("name"); String email =
    request.getParameter("email"); String pass =
    request.getParameter("password"); String gender =
```

```

request.getParameter("gender");

if (name != null && email != null && pass != null && gender != null &&
!name.isEmpty() && !email.isEmpty() && !pass.isEmpty()) { try {
    PreparedStatement ps = con.prepareStatement(
        "INSERT INTO users(name, email, password, gender) VALUES (?, ?, ?, ?)");
    ps.setString(1, name);
    ps.setString(2,
        email);
    ps.setString(3,
        pass);
    ps.setString(4, gender);
    int rows = ps.executeUpdate();
    msg = rows > 0 ? "Registered successfully!" : "Registration failed."; msgClass =
        rows > 0 ? "success" : "error";
    ps.close();
} catch (Exception e) {
    msg = "Error: " + e.getMessage();
    msgClass = "error";
}
} else {
    msg = "All fields are required.";
    msgClass = "warn";
} %>
<div class="container">
    <h2>Register</h2>

    <form method="post">
        <div class="field">
            <label>Full Name</label>
            <input type="text" name="name" required>
        </div>
        <div class="field">
            <label>Email</label>
            <input type="email" name="email" required>
        </div>
        <div class="field">
            <label>Password</label>
            <input type="password" name="password" required>
        </div>
        <div class="field">
            <label>Gender</label>
            <div class="radio-group">
                <label><input type="radio" name="gender" value="Male" required> Male</label>
                <label><input type="radio" name="gender" value="Female"> Female</label>
                <label><input type="radio" name="gender" value="Other"> Other</label>
            </div>
        </div>
        <button type="submit">Register</button>
    </form > </div>

```

Login.jsp

```
<%@ page import="java.sql.*" %>
<%@ include file="db.jsp" %>

<%
String msg = "";
String msgClass = "";
if ("POST".equalsIgnoreCase(request.getMethod())) { String email =
    request.getParameter("email");
    String password = request.getParameter("password");

    if (email != null && password != null && !email.isEmpty() && !password.isEmpty()) { try {
        PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE email=? AND
password=?");
        ps.setString(1, email); ps.setString(2,
password); ResultSet rs =
ps.executeQuery();

        if (rs.next()) {
            session.setAttribute("user", rs.getString("name"));
            session.setAttribute("userId", rs.getInt("id"));
            response.sendRedirect("dashboard.jsp");
        } else {
            msg = "Invalid email or password."; msgClass =
            "error";
        }
        rs.close(); ps.close();
    } catch (Exception e) {
        msg = "Error: " + e.getMessage(); msgClass =
        "error";
    }
} else {
    msg = "All fields are required."; msgClass = "warn";
}
}
%>
<div class="container">
    <h2>Login</h2>

    <form method="post">
        <div class="field">
            <label>Email</label>
            <input type="email" name="email" required>
        </div>
        <div class="field">
            <label>Password</label>
            <input type="password" name="password" required>
        </div>
        <button type="submit">Login</button>
    </form>
</div>
```

Dashboard.jsp

```
<%@ page import="java.sql.*" %>
```

```

<%@ include file="db.jsp" %>

<%
if (session.getAttribute("user") == null) {
    response.sendRedirect("login.jsp"); return;
}

String loggedUser = (String) session.getAttribute("user");

// Handle delete
String msg = "";
if ("POST".equalsIgnoreCase(request.getMethod())) { String delId =
    request.getParameter("delId");
    if (delId != null) { try {
        PreparedStatement ps = con.prepareStatement("DELETE FROM users WHERE id=?"); ps.setInt(1, Integer.parseInt(delId));
        ps.executeUpdate(); ps.close();
        msg = "User deleted.";
    } catch (Exception e) {
        msg = "Error: " + e.getMessage();
    }
}
}
%>

```

Output:

Register

Full Name

Email

Password

Gender

☒ Male ☐ Female ☐ Other

Register

Already have an account? [Login](#)

Login

Email

Password

Login

No account? [Register](#)

Welcome, Sushank Lamsal

All Users

Showing all registered users

ID	Name	Email	Gender	Action
1	Sushank Lamsal	sushanklamsal@gmail.com	Male	Delete
2	Alisha Bhandari	alishabhandari@gmail.com	Female	Delete
3	Liam Thapa	liamthapa99@gmail.com	Male	Delete